UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

Registradores, Portas de Entrada e Saída, Configurações Básicas do PIC18F4550 e Programação C

Docente: Prof. Dr. Ruhan Pontes Policarpo de Souza
Discente: Carlos Gabriel Baratieri — 2706598
Discente: Luiz Fernando Coimbra Hintz — 2559145
Turma: C51B

O presente documento tem como objetivo relatar a primeira prática laboratorial da disciplina de Sistemas Microcontrolados, envolvendo registradores e portas de entrada e saída, configurações básicas do PIC18F4550.

Sistemas Microcontrolados — EC45C
08/09/2026

13 de setembro de 2026 — 20:59
Discente: Carlos Gabriel Baratieri
Discente: Luiz F. Coimbra Hintz

1 Introdução

O presente experimento tem como objetivo abordar o funcionamento das portas de entrada e saída do microcontrolador PIC18F4550 por meio de ferramentas apresentadas pelo professor em aulas anteriores, tais como o Proteus e o compilador MPLAB X IDE, além de fazer uso do kit didático XM118 – Exsto e do PICkit 3 da Microchip.

Dessa forma, foi criado um esquemático usando a ferramenta ISIS do Proteus, como será visto mais para frente no decorrer do relatório, para testarmos um firmware desenvolvido no laboratório, com foco em leitura e escrita de portas de I/O, simular o firmware no Proteus e implementar no kit didático, realizando a gravação e execução do código de máquina no microcontrolador.

Basicamente, o firmware era um contador binário, mas por meio de incrementos e decrementos, ou seja, ele tinha como função contar de 0 até 15, mas em representação binária, e visualizávamos tal representação por meio dos LEDs sendo acesos e apagados. O incremento ou decremento ocorreria por meio do pressionamento de botões específicos, como será visto mais para frente, cada incremento ou decremento era unitário. Além disso, tínhamos uma ação de zerar os LEDs, e os incrementos e decrementos possuíam limites que serão vistos também no decorrer dos códigos.

Esse firmware foi simulado no Proteus e no kit didático XM118 – Exsto. Apenas um adendo: o kit possuía uma lógica de estados no LED meio que inversa, mas isso será abordado no decorrer do laboratório. Em sala de aula, em momentos anteriores, o professor explicou o motivo disso, tanto que ele usou isso para ensinar sobre inversores lógicos. Mas, independentemente do estado do LED, isso não interfere no conceito lógico.

2 Materiais e Métodos

Os materiais utilizados para a realização do experimento foram o microcontrolador PIC18F4550, o kit didático XM118 – Exsto (com PIC18F4550 integrado), PICkit 3 da Microchip, um computador com o MPLAB X IDE versão 5.20, o software Proteus para simulação e cabos para conexão entre os equipamentos.

O PICkit 3 foi utilizado para realizar a gravação do firmware no microcontrolador presente no kit didático XM118. A conexão entre o programador e o kit pode ser observada na Figura 1.

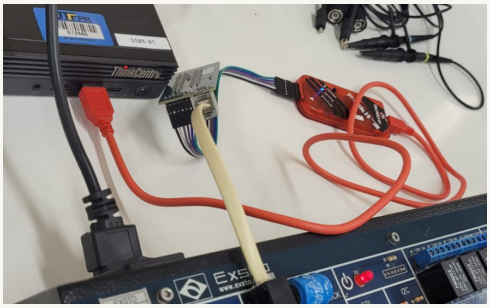


Figura 1: Programador PICkit 3 conectado ao kit didático XM118.

O kit didático XM118, utilizado durante a realização do experimento, é apresentado na Figura 2.



Figura 2: Kit didático XM118 – Exsto.

Para o desenvolvimento e compilação do firmware, foi utilizado o MPLAB X IDE versão 5.20, conforme apresentado na Figura 3.

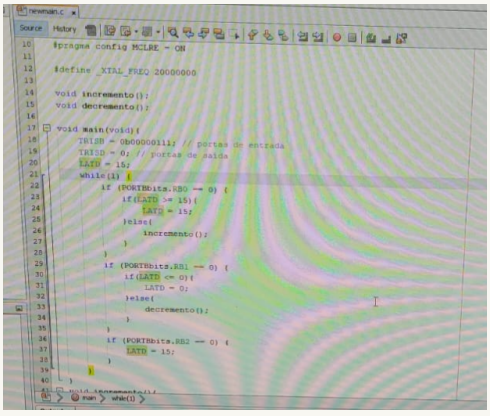


Figura 3: MPLAB X IDE com o código-fonte do firmware.

Além disso, o software Proteus foi utilizado para a criação do esquemático e para a simulação do circuito, para verificar o funcionamento do firmware antes de sua implementação no kit didático. A interface do software pode ser observada na Figura 4.

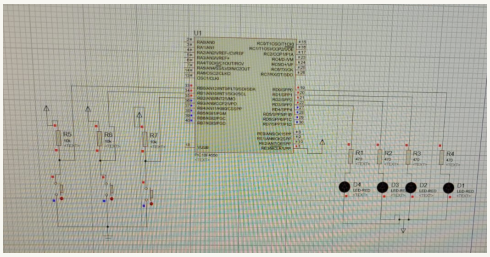


Figura 4: Software Proteus utilizado para a criação e simulação do circuito.

Abaixo, podemos observar o esquemático realizado no Proteus, apresentado na Figura 5.

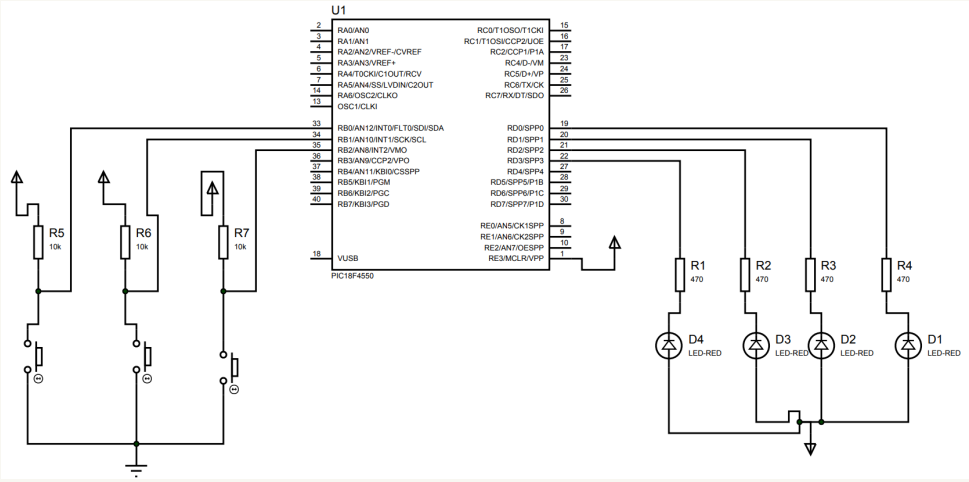


Figura 5: Esquemático do circuito desenvolvido no Proteus.

Durante o experimento, a metodologia utilizada para a realização do laboratório consistiu, inicialmente, na leitura do roteiro disponibilizado pelo professor em formato PDF, acompanhada da atenção às explicações referentes ao tema abordado.

Logo após, foi utilizado o software Proteus, por meio da ferramenta ISIS, para a elaboração do esquemático do experimento, conforme apresentado no roteiro. Essa etapa teve como objetivo permitir a realização de testes do código no ambiente de simulação, possibilitando a reprodução dos testes mesmo sem a utilização do kit didático. O esquemático desenvolvido no Proteus pode ser observado na Figura 5.

Após essa etapa, foi utilizado o compilador MPLAB X IDE versão 5.20 para realizar a escrita do código em linguagem C, permitindo a execução do experimento. O código foi desenvolvido de acordo com o enunciado proposto, que consistia em:

> Implemente um sistema que acenda e apague os LEDs ligados às portas RD0, RD1, RD2 e RD3, de modo a representar um número binário de 4 bits. Em seguida, incremente, decremente e zere esse número binário (4 LEDs) por meio dos botões conectados em SW4, SW3 e SW2.

Em resumo, o experimento consistia em construir um contador binário de 4 bits, capaz de representar valores de 0 a 15. O contador deveria permitir a incrementação e a decrementação do valor representado pelos LEDs, além de possuir uma opção para zerar os bits.

3 Resultados

Abaixo está o código que foi testado no kit. No Proteus talvez ocorram pequenas mudanças, mas é em questão do estado do LED, pois no kit o estado zero, que normalmente é desligado, os LEDs ficam ligados. No Proteus é o oposto, o estado zero é de fato o desligado, logo existe essa pequena diferença, mas tudo pode ser mudado facilmente no LATD.

```c
#include <xc.h>

// Configurações
#pragma config PLLDIV = 5
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

#define _XTAL_FREQ 20000000

void incremento();
void decremento();

void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;

    while(1) {
        if (PORTBbits.RB0 == 0) {
            if (LATD >= 15) {
                LATD = 15;
            } else {
                incremento();
            }
        }

        if (PORTBbits.RB1 == 0) {
            if (LATD <= 0) {
                LATD = 0;
            } else {
                decremento();
            }
        }

        if (PORTBbits.RB2 == 0) {
            LATD = 15;
        }
    }
}

void incremento() {
    LATD++;
    __delay_ms(300);
}

void decremento() {
    LATD--;
    __delay_ms(300);
}
```

Basicamente, a lógica do código é a seguinte. Primeiro, definimos a biblioteca, as configurações iniciais e as macros, nas quais usamos como modelo os códigos das aulas passadas. Como pode ser visto abaixo.

```c
// Biblioteca
#include <xc.h>

// Configurações
#pragma config PLLDIV = 5
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

// Macro
#define _XTAL_FREQ 20000000
```

Logo após tudo isso, ao olharmos o enunciado, vimos que teríamos que realizar incremento e decremento de alguma coisa, logo, fizemos funções e suas prototipações.

```
void incremento();
void decremento();

// [...]

void incremento() {
}

void decremento() {
}
```

Depois fomos configurar as portas de entrada e saída, assim como o estado inicial. Lembrando, esse código foi feito para o kit, logo, está invertido. Para o Proteus é apenas trocar de 15 para 0. Mas isso não muda o conceito lógico do código. Podemos observar essa inversão nos LEDs por meio da Figura 2.

```
void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;          // Estado inicial (kit)
//  [...]
}
```

Por convenção padrão de inserção de código, realizamos um looping infinito, para que o microcontrolador fique verificando os comandos, no nosso caso, os botões.

```
void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;          // Estado inicial (kit)

    while(1) {
//  [...]
    }
}
```

Depois, escrevemos os esqueletos das condicionais. Basicamente, elas têm o papel de verificar quando os botões foram selecionados, seguindo literalmente a lógica: *"se esse botão for pressionado, executará essa ação"*. Porventura, a condição também verifica se o valor já atingiu o limite máximo.

Para a primeira condicional verificamos se `SW4` foi pressionado, por meio do `RB0`. Primeiro verificamos se estamos no limite máximo, se estivermos ele permanece no limite de um número binário representado por 4 bits (0b1111), como dito no enunciado, caso contrário ele chama a função de `incremento()`.

```
void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;

    while(1) {
        if (PORTBbits.RB0 == 0) {
            if (LATD >= 15) {
                LATD = 15;
            } else {
                incremento();
            }
        }
}
```

A função de `incremento()` quando é chamada executa o comando `LATD++` faz com que o valor armazenado em `LATD` seja aumentado em 1. O `delay` foi usado apenas para controle de velocidade nos LEDs.

```
void incremento() {
    LATD++;
    __delay_ms(300);
}
```

A segunda condicional verifica se o `SW3`, conectado ao `RB1` foi pressionado. Primeiro ele verifica se estamos no impede que o valor ultrapasse o limite mínimo, caso contrário ele realiza a ação de `decremento()`

```
void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;

    while(1) {
        if (PORTBbits.RB0 == 0) {
            if (LATD >= 15) {
                LATD = 15;
            } else {
                incremento();
            }
        }

        if (PORTBbits.RB1 == 0) {
            if (LATD <= 0) {
                LATD = 0;
            } else {
                decremento();
            }
        }
}
```

A função de `decremento()` quando é chamada executa o comando `LATD--` faz com que o valor armazenado em `LATD` seja diminuído em 1. O `delay` foi usado apenas para controle de velocidade nos LEDs.

```
void decremento() {
    LATD--;
    __delay_ms(300);
}
```

Por fim, verificamos o `SW2`, conectado ao `RB2`. Quando pressionado, `LATD` recebe 15, que no kit ele deixa tudo zerado. Novamente, no Proteus seria 0.

```
void main(void) {
    TRISB = 0b00000111; // Portas de entrada
    TRISD = 0;          // Portas de saída
    LATD = 15;

    while(1) {
        if (PORTBbits.RB0 == 0) {
            if (LATD >= 15) {
                LATD = 15;
            } else {
                incremento();
            }
        }

        if (PORTBbits.RB1 == 0) {
            if (LATD <= 0) {
                LATD = 0;
            } else {
                decremento();
            }
        }

        if (PORTBbits.RB2 == 0) {
            LATD = 15;
        }
}
```

Foi usado o `PORTBbits.RXY` pois segundo o material do professor eles permitem acesso de apenas um bit do registrador `PORT`, onde o `x` seria nome da porta representado por letra maiúscula e o `y` seria o nome do pino, assim, cada x e y usado tem relação com o diagrama da Figura 5. O `PORT` escreve ou realiza leitura sobre os pinos associados à porta. Foi usado `TRIS` pois segundo o material do professor configura o sentido do fluxo de dados de uma determinada porta e o `LAT` armazena o valor do último comando de escrita.

O kit teve que ser trocado uma única vez, pois o primeiro kit estava em curto-circuito em relação aos botões, mas ele foi trocado posteriormente, e foi um sucesso a implementação e o teste do firmware.

4 Conclusão

Portanto, neste experimento, foi abordada a implementação de um firmware para o microcontrolador PIC18F4550, com foco em suas entradas e saídas. Foi simulado o teste do firmware por meio do Proteus e realizado um teste prático no kit didático XM118 – Exsto. Além disso, o firmware foi compilado usando o MPLAB X IDE 5.20, e foi utilizado o PICkit 3 da Microchip.

O firmware apresentou bom funcionamento, realizando incrementos, decrementos e o zeramento do contador binário, além da lógica das limitações. Após a troca do kit, que apresentou problemas nos botões, foi possível realizar a implementação e os testes do firmware com sucesso. Dessa forma, os objetivos propostos para o experimento foram alcançados.